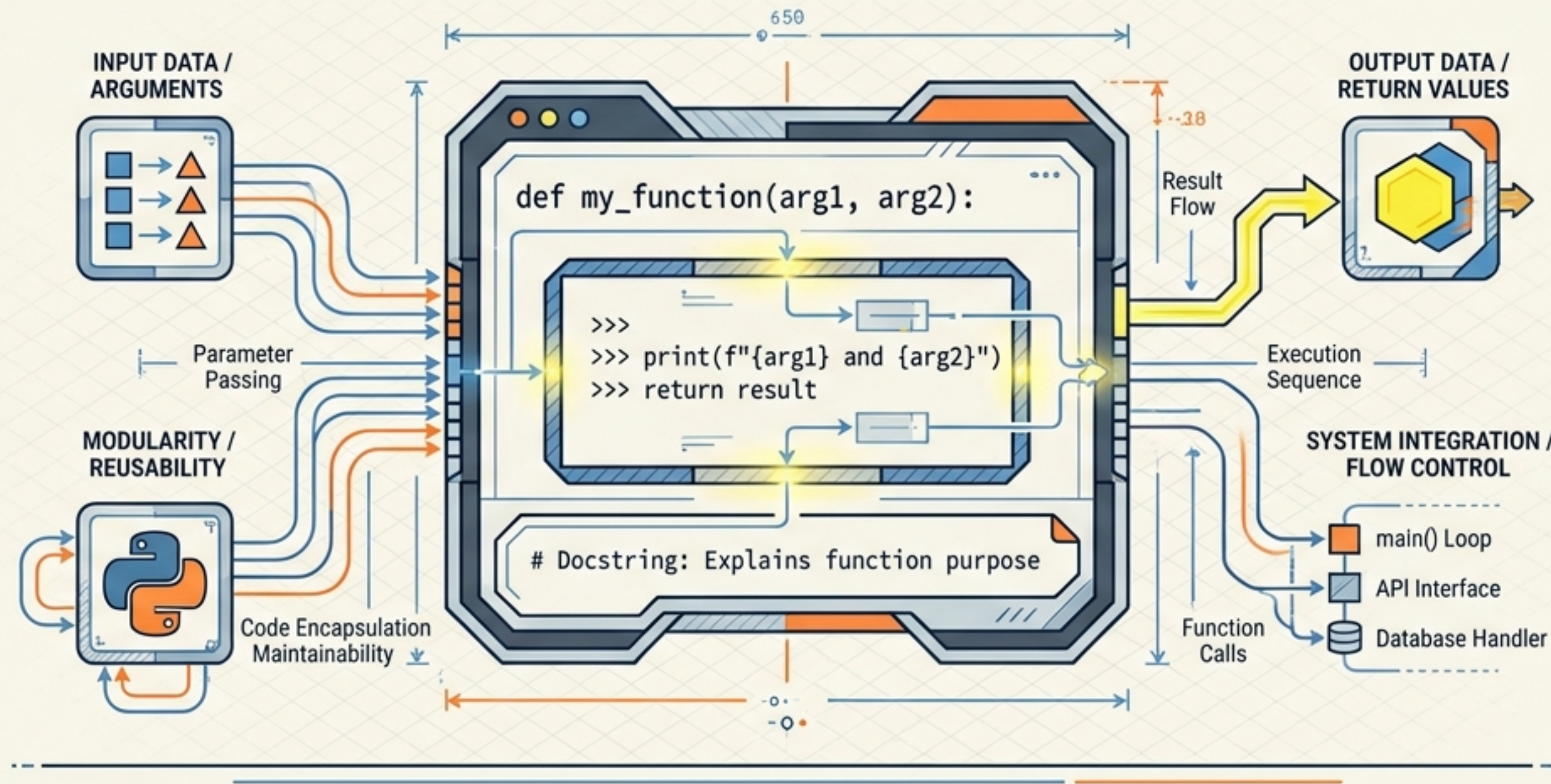




Funções em Python

Dominando a modularidade, o fluxo de controle e a reutilização de código.

Sumário da Arquitetura

1. O Conceito de Função



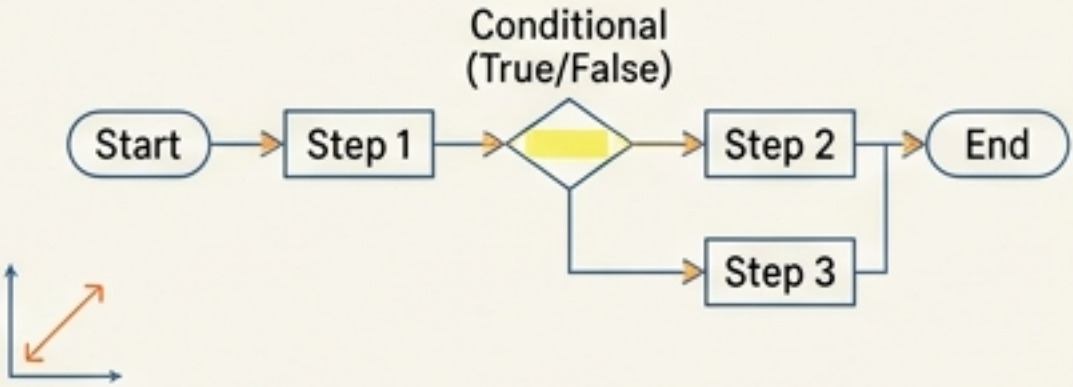
2. Anatomia e Sintaxe

```
def function_name(parameters):  
    # corpo da função
```

Corpo da função

Placehadores

3. O Fluxo de Controle



4. Entradas: Parâmetros e Argumentos

Parâmetros (Definição)

Argumentos (Chamada)

```
def saudacao(nome, idade):  
    saudacao("Ana", 30)
```

5. Tipos de Argumentos Avançados

Posicionais: func(a, b)
Palavra-chave: func(key=value)
Padrão: def func(a=10):
Variáveis: func(*args, **kwargs)

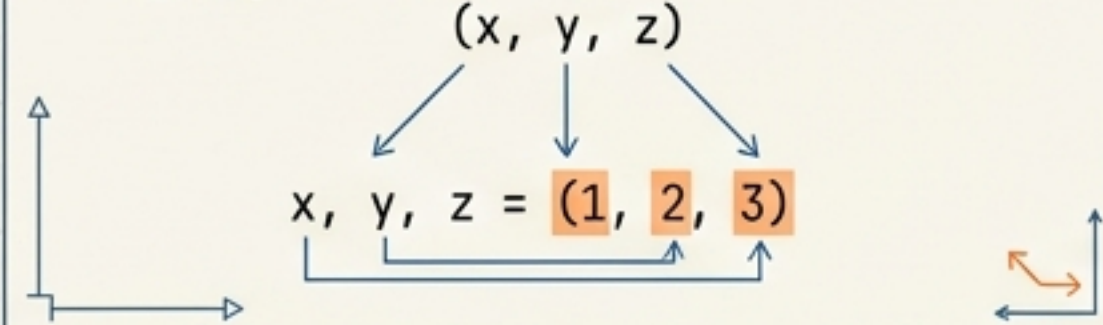
6. Saídas: Retornando Valores

Return Statement

Resultado

```
return resultado
```

7. Desempacotamento (Tuplas)



8. Boas Práticas (Docstrings)

```
def my_function():  
    """Documentação da função."""
```

Descrição

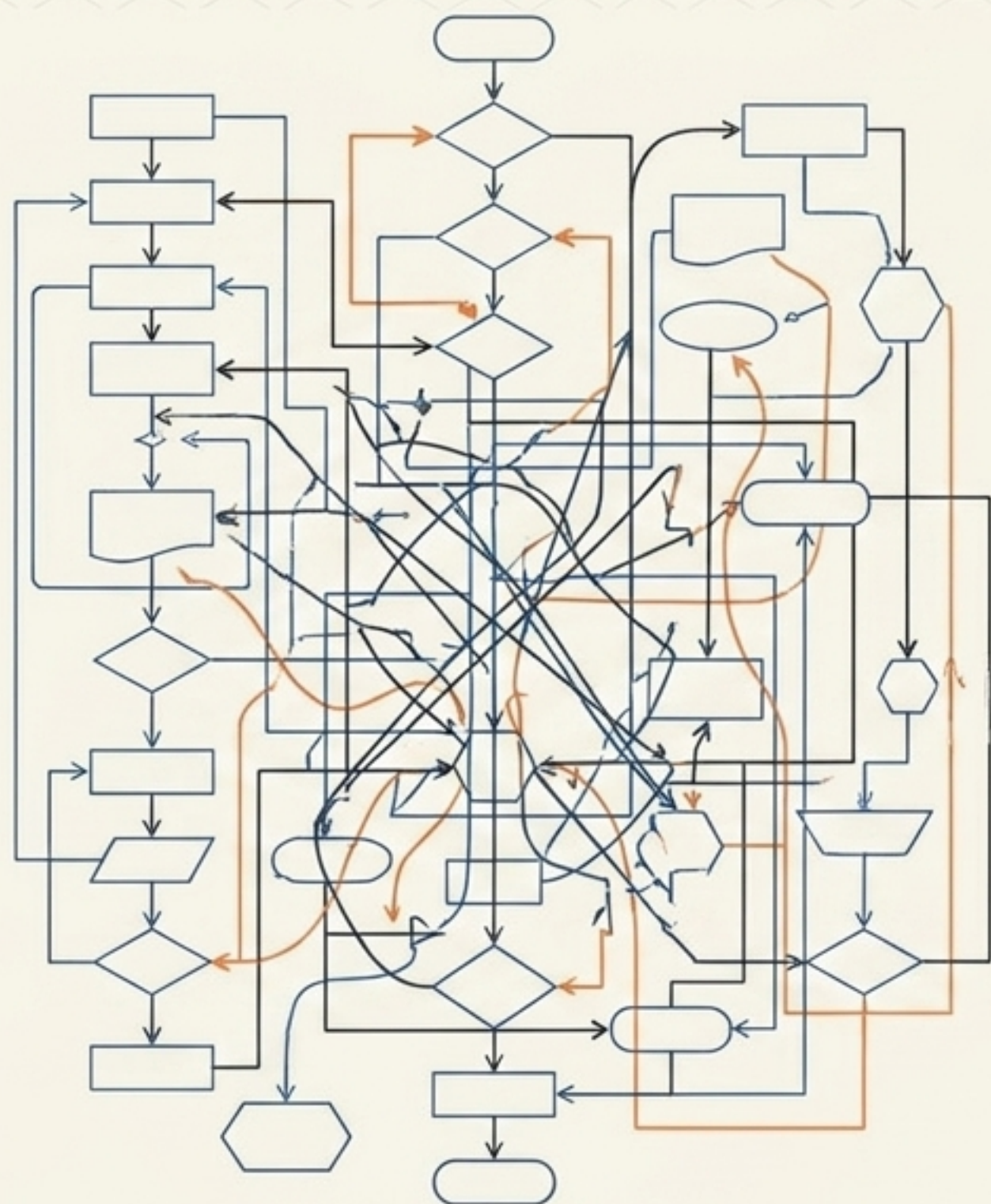
Metadiização

Metadados

9. Funções Anônimas (Lambda)

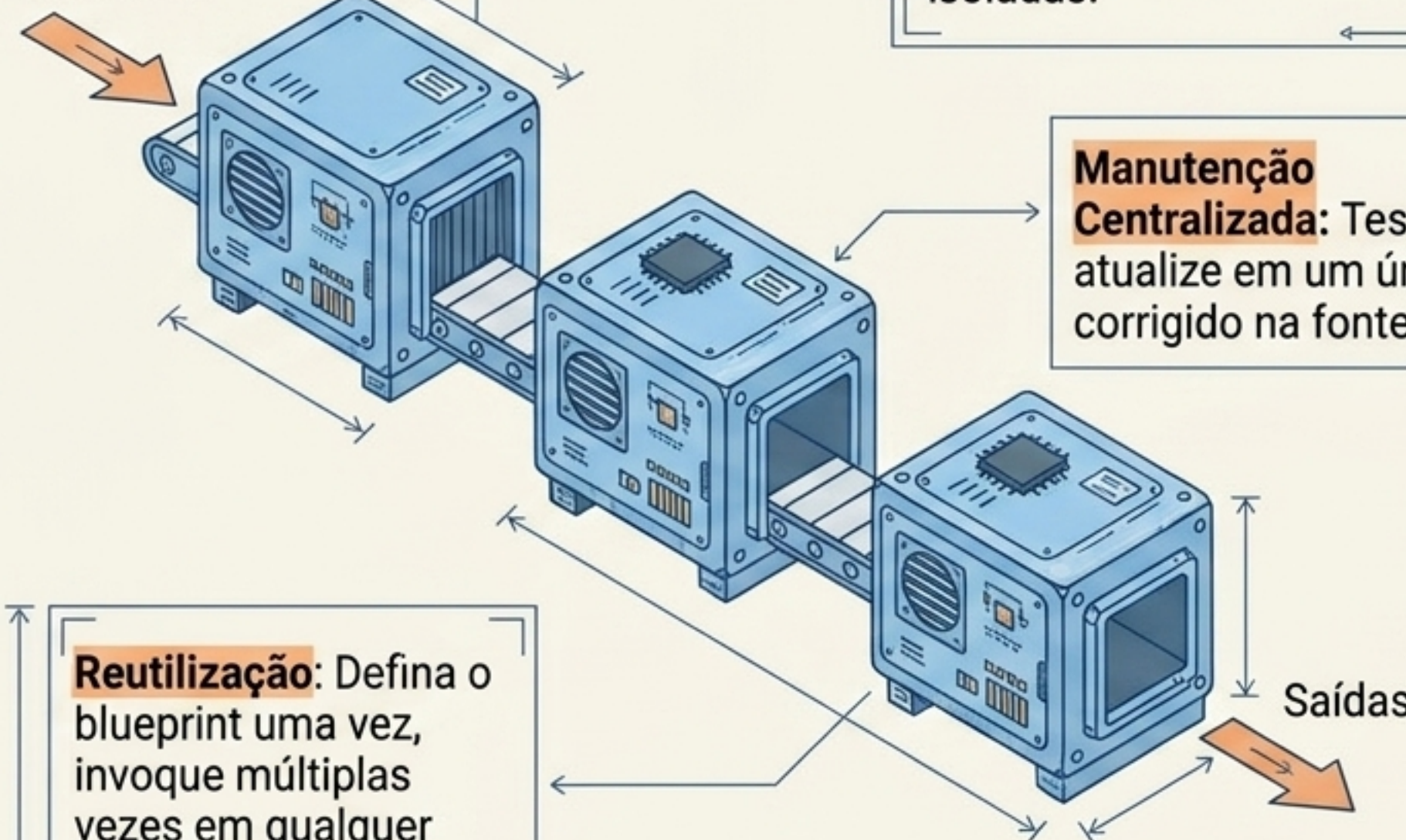
```
lambda arguments: expression
```

soma = lambda x, y: x + y



Programa Monolítico

Entradas

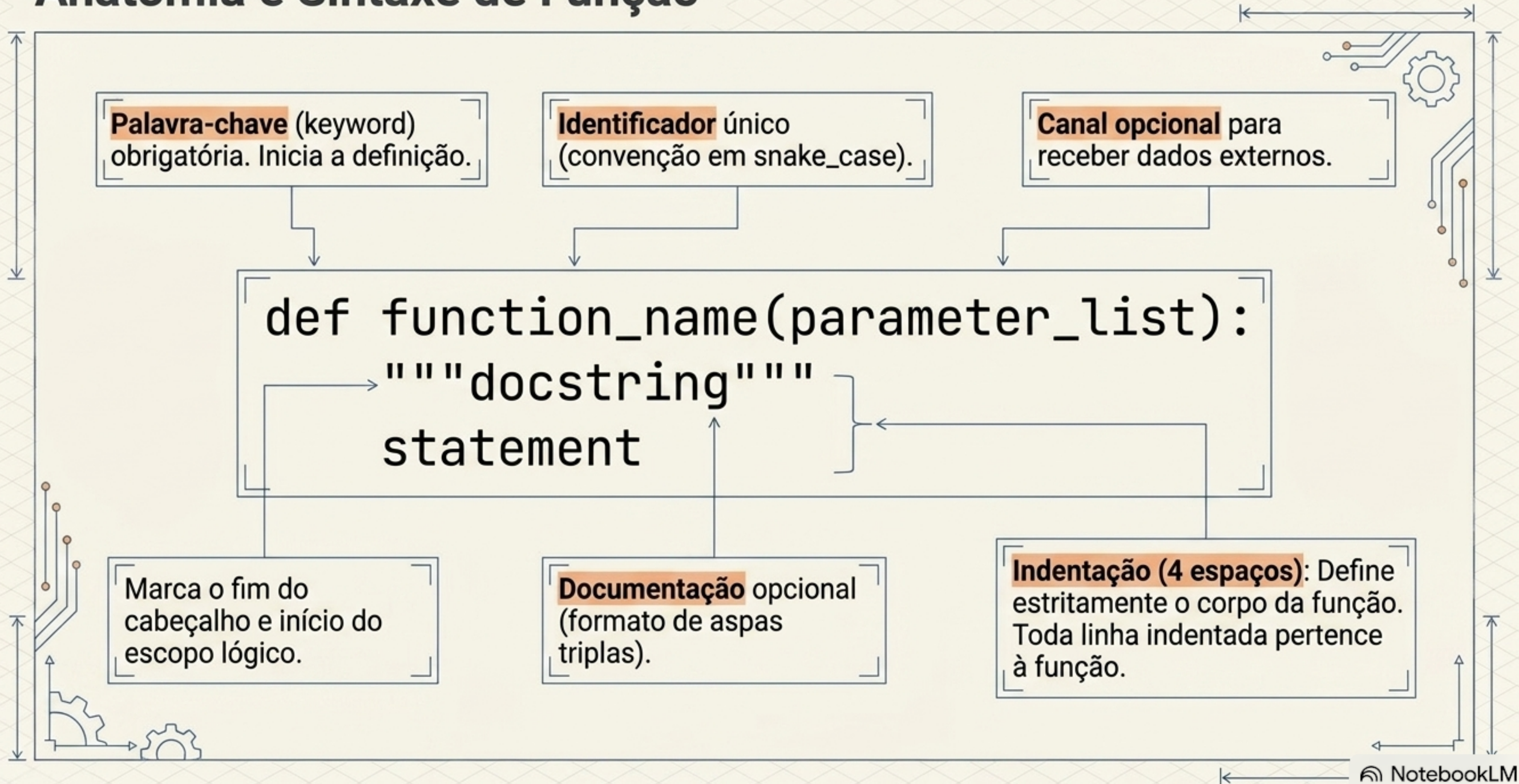


Modularidade: Quebra aplicações complexas em unidades lógicas e isoladas.

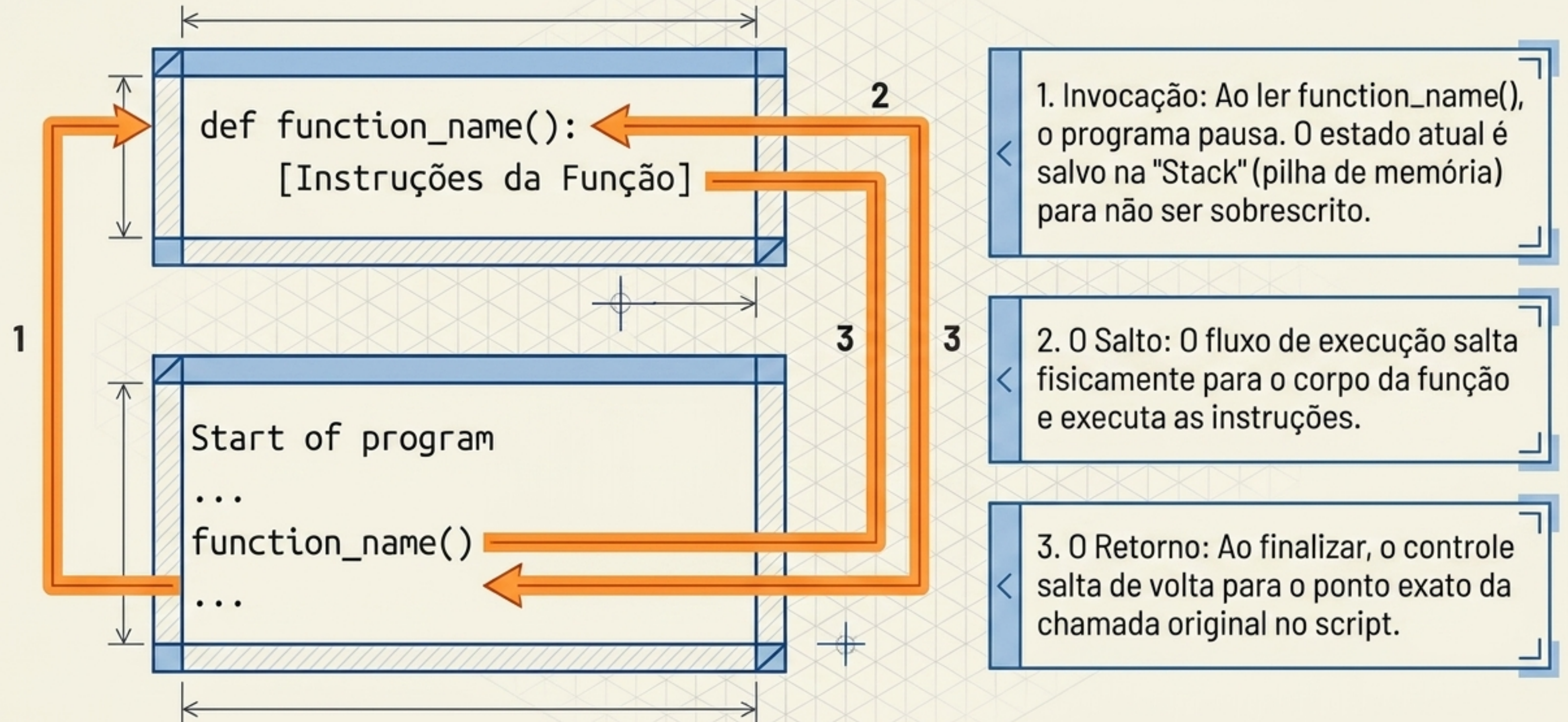
Manutenção Centralizada: Teste e atualize em um único. corrigido na fonte.

Reutilização: Defina o blueprint uma vez, invoque múltiplas vezes em qualquer parte do código.

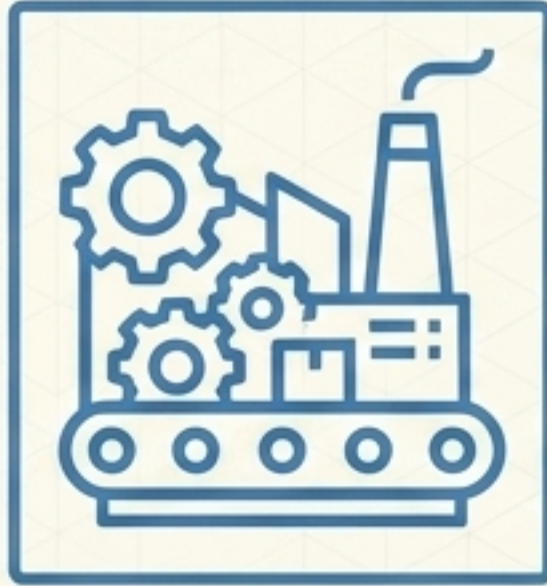
Anatomia e Sintaxe de Função



O Fluxo de Controle (O 'Salto')



Nativas (Built-in)



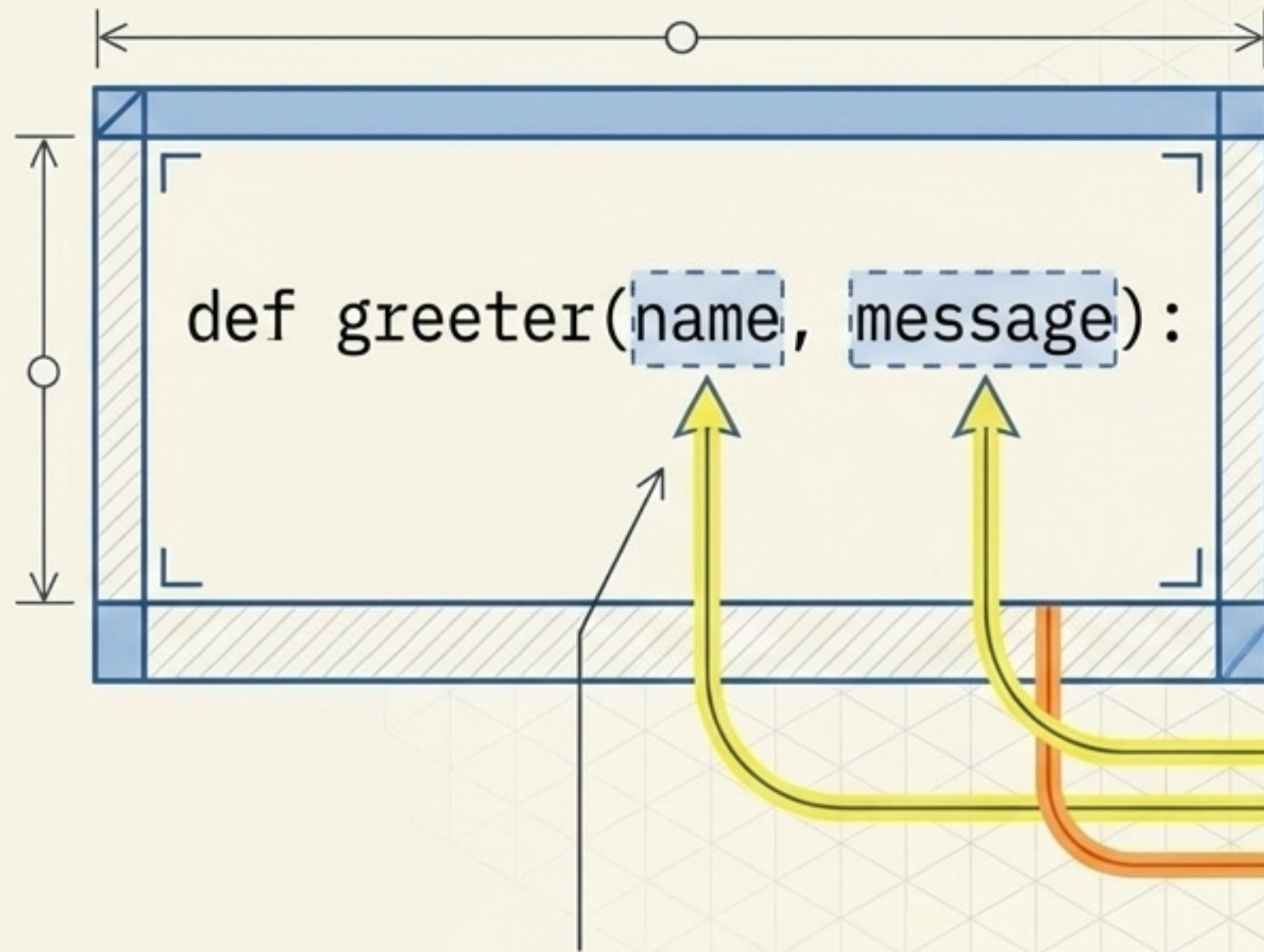
- Fornecidas pelo próprio núcleo da linguagem Python.
- Prontas para uso imediato, não requerem declaração **def**.
- Exemplos: **print()**, **input()**.

Definidas pelo Usuário (User-defined)

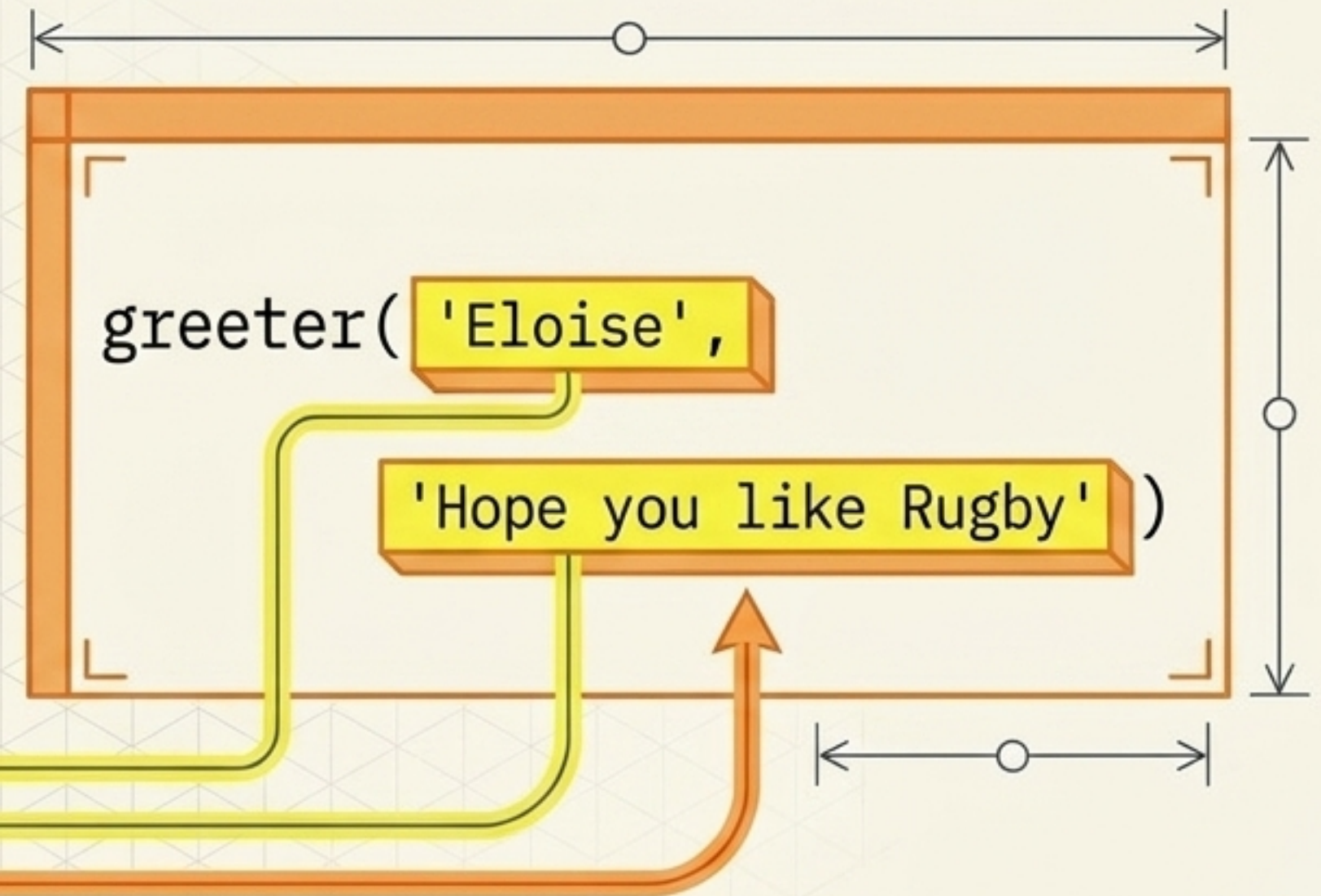


- Escritas pelo desenvolvedor para resolver problemas de domínio específico.
- Requerem declaração explícita com **def** e uma assinatura própria.
- Exemplos: **print_my_msg()**, **square(n)**.

A Planta (Definição)



O Material (Chamada)



Parâmetro: A variável no cabeçalho. Funciona como um "slot" vazio aguardando dados.

Argumento: O valor real ou dado concreto passado no momento da invocação.

Flexibilidade: Valores Padrão (Default)

The Definition

```
def greeter(name, message='Live Long and Prosper'):
```

The Invocations

Path A	<code>greeter('Eloise')</code>		Welcome Eloise - Live Long and Prosper
Path B	<code>greeter('Eloise', 'Hope you like Python')</code>		Welcome Eloise - Hope you like Python

Atribuir um valor na definição torna o argumento opcional.

Se omitido, o Python injeta o valor padrão. Se fornecido, o novo argumento sobrescreve o padrão.

Regra de Ouro: Parâmetros com valores padrão devem obrigatoriamente ficar à direita de parâmetros obrigatórios na definição.

Flexibilidade: Argumentos Nomeados (Keyword)

A Definição

```
def greeter(name, message):
```

A Invocação

```
greeter(message='We like Python', name='Lloyd')
```

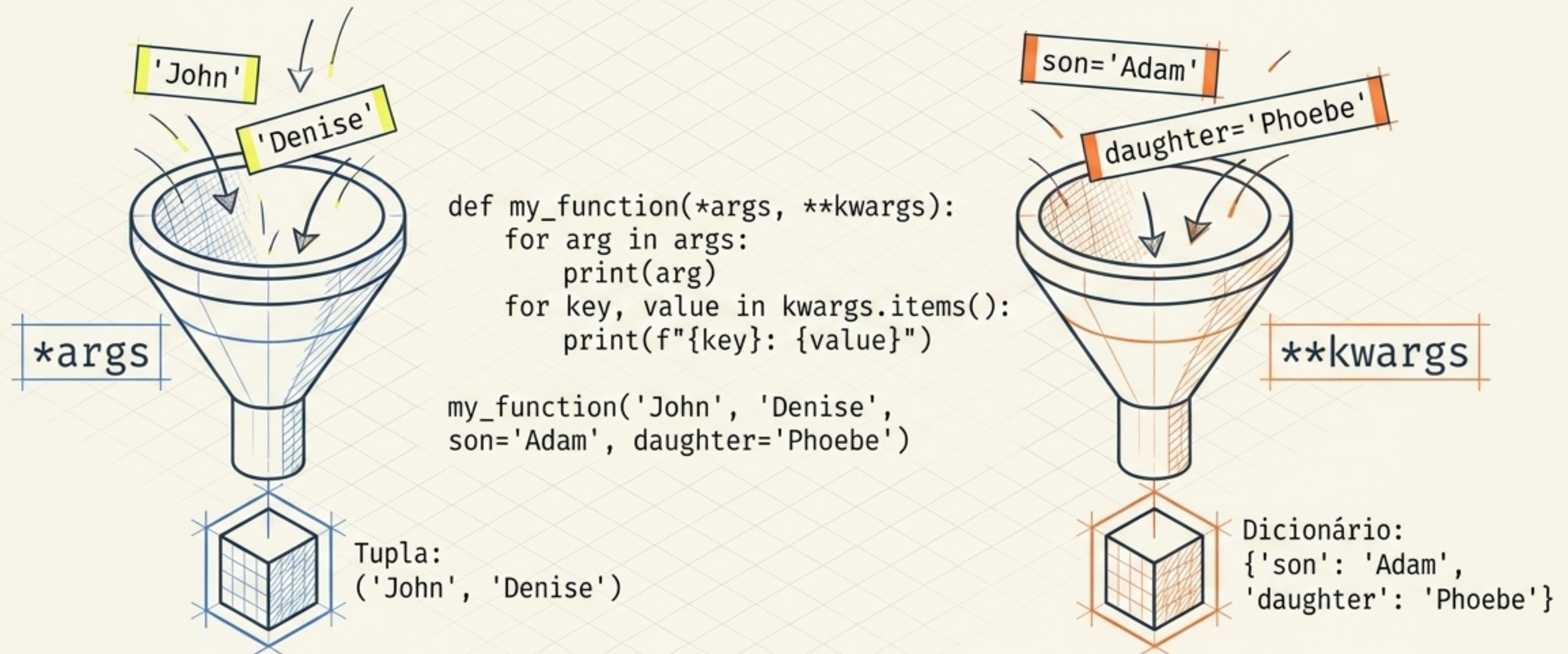
Ao nomear o argumento na invocação, você elimina a necessidade de seguir a ordem posicional. Ideal para funções com múltiplos parâmetros opcionais.



Atenção: Argumentos posicionais nunca podem vir após um argumento nomeado na chamada do código.

Argumentos Arbitrários (*args e **kwargs)

Usados quando não se sabe exatamente quantos argumentos serão fornecidos.



Empacota múltiplos argumentos posicionais soltos.

Empacota múltiplos argumentos nomeados em pares de chave-valor.

Saídas — A Instrução return

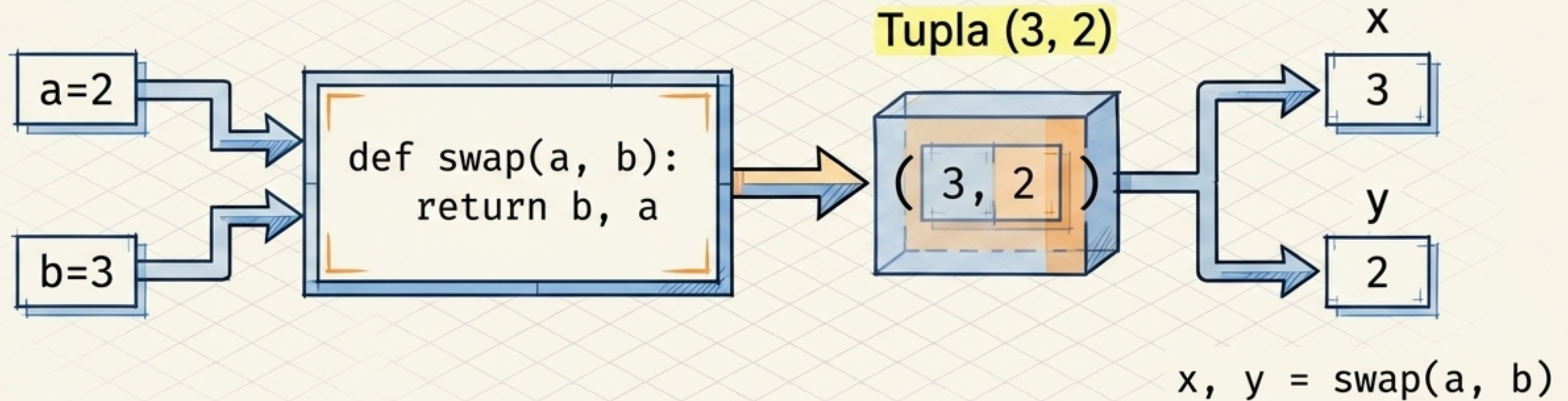
```
def square(n):  
    return n * n
```

The diagram illustrates the execution of the `square` function. A blue line connects the `return` statement in the function definition to a box containing the value `16`. From this box, another blue line with an arrow points to the argument `4` in the function call `square(4)` within the `result = square(4)` statement, indicating that the function's output is being passed to its input.

```
result = square(4)
```

- A instrução `return` finaliza a função imediatamente e devolve o valor calculado ao escopo chamador.
- O valor retornado pode ser armazenado em variáveis, passado para outras funções ou usado em expressões lógicas.
- Nota: Se uma função não possui `return`, o Python implicitamente retorna `None`.

Retornando Múltiplos Valores (Tuplas)



- O Python permite retornar vários valores separando-os por vírgula no return.
- Nos bastidores: O Python agrupa (empacota) esses dados em uma **Tupla** única.
- Desempacotamento Elegante: Na chamada, você pode atribuir a saída diretamente a múltiplas variáveis em uma única linha.

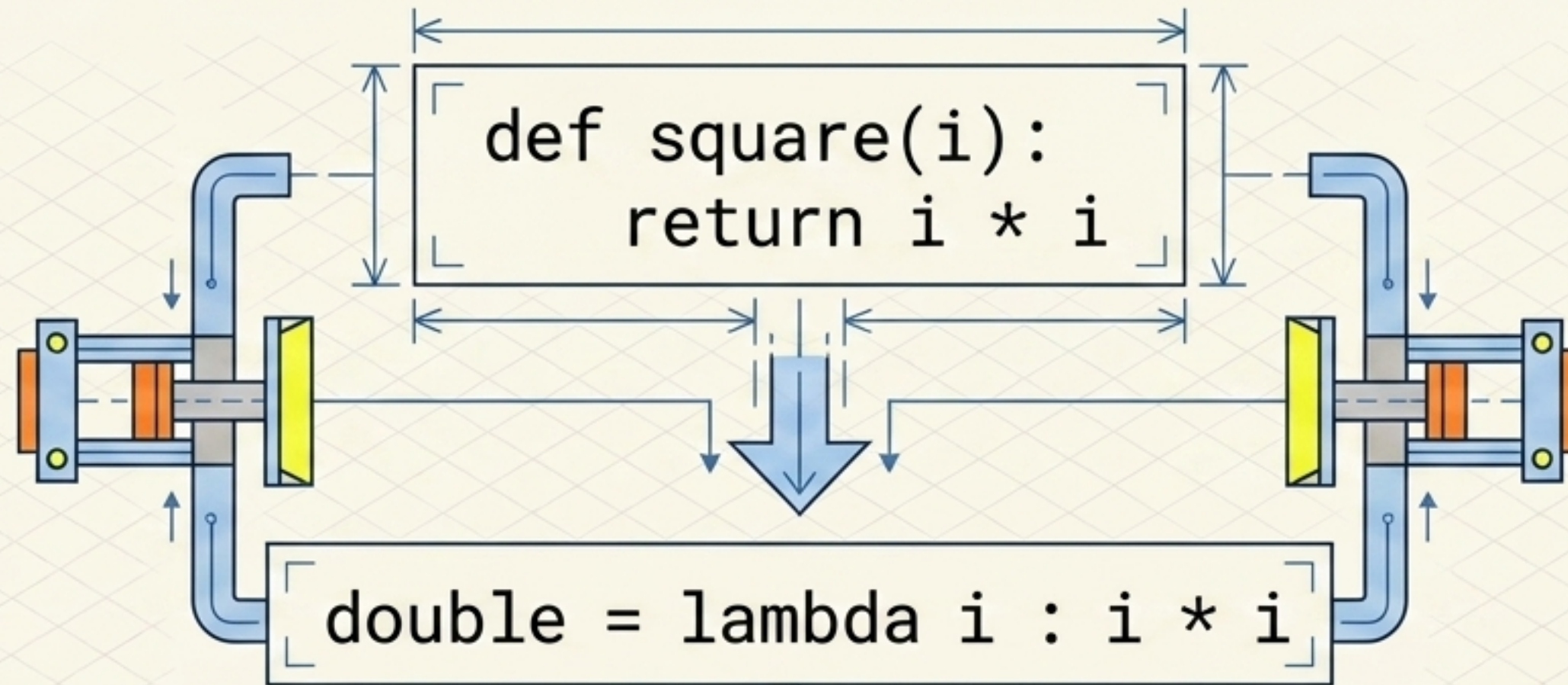
Boas Práticas: Docstrings

```
def get_integer_input(message):  
    """This function will display the message to the  
    user and request that they input an integer."""
```

This function will display the message to the user and request that they input an integer.

- Strings de aspas triplas logo após o cabeçalho documentam o propósito e os parâmetros esperados.
- Lidas automaticamente por IDEs para fornecer assistência em tempo real.
- Acessíveis nativamente no código através do atributo especial `__doc__`.

Funções Anônimas (Expressões lambda)



Sintaxe: `lambda [argumentos]: [expressão de retorno]`

- Funções sem nome para uso descartável.
- Restrição: Contêm apenas uma única expressão (não permitem múltiplos blocos). O resultado é retornado automaticamente.

Blueprint Resumo: Funções

Estrutura Básica

- Palavra-chave `def`, parênteses `()`, dois-pontos `:`, indentação estrutural.

Comportamento

- O fluxo 'salta' para a definição, salva o estado na Pilha (Stack), executa o corpo e retorna.

Entradas (Parâmetros)

- **Posicionais:** Ordem rígida.
- **Padrão (Default):** Tornam o argumento opcional.
- **Nomeados (Keyword):** Ignoram a ordem posicional.
- **Arbitrários:** `*args` (Tuplas), `**kwargs` (Dicionários).

Saídas

- A instrução `return` encerra a execução e devolve valores (únicos ou múltiplos via Tuplas).

Avançado

- Expressões `lambda` para funções anônimas de linha única.